

# Factory Compliance & Alert Escalation System

## Master Build Guide

*Step-by-Step Tutorial & Tech Stack Reference*

Prepared for: Genesys Research Lab — Intern Take-Home Assessment  
Suggested build window: 3 days

## Table of Contents

## 1. Project Overview

This system watches short factory security-camera clips, reads the facility's OHS Compliance Policy Manual to learn what counts as a violation, finds those violations in the video, decides how serious each one is, routes it to the right channel, writes a permanent record, and displays everything on a dashboard. Five modules make this up, and they all feed into one another rather than working in isolation.

- Module 1 — Detection Engine: finds violations in video, grounded in rules extracted from the policy PDF.
- Module 2 — Severity Matrix: assigns LOW / MEDIUM / HIGH / CRITICAL to each detected violation.
- Module 3 — Escalation Pipeline: routes LOW/MEDIUM to a database log, and HIGH/CRITICAL to a database log plus a real-time alert.
- Module 4 — Report Generation: writes an immutable, structured record for every violation.
- Module 5 — Dashboard: a live feed view, an alert timeline, and a filterable historical log with export.

*The four behavior classes this system must recognize are: Safe Walkway Violation, Unauthorized Intervention, Opened Panel Cover, and Carrying Overload with Forklift.*

## 2. Tech Stack & Libraries

Everything below stays inside a single Python repository. The choices favor speed of delivery over architectural purity, which matters given the short deadline — there is no need for a separate frontend framework, a managed database server, or custom-trained models.

### 2.1 Core Stack at a Glance

| Layer            | Library / Tool       | Install                   | Purpose                                                                                      |
|------------------|----------------------|---------------------------|----------------------------------------------------------------------------------------------|
| Language         | Python 3.10+         | —                         | Single language across the whole pipeline; simplest to wire together quickly.                |
| Video I/O        | opencv-python        | pip install opencv-python | Open clips, extract frames, draw overlays, simple color masking for the vest/walkway checks. |
| Object detection | ultralytics (YOLOv8) | pip install ultralytics   | Pretrained person / vehicle detection — no custom training needed.                           |

|                           |                           |                                                 |                                                                                                                                                                           |
|---------------------------|---------------------------|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Vision-language reasoning | google-genai (Gemini API) | pip install google-genai                        | Judging panel open/closed state and block counts on cropped frames; also used for policy parsing, with optional structured JSON output via <code>response_schema</code> . |
| PDF parsing               | pdfplumber                | pip install pdfplumber                          | Extract raw text from the policy manual before rule extraction.                                                                                                           |
| Schema validation         | pydantic                  | pip install pydantic                            | Enforce the rule schema and report schema so malformed LLM output is caught early.                                                                                        |
| Storage                   | sqlite3 (built-in)        | —                                               | Single-file persistent log; zero setup, easy to filter and export.                                                                                                        |
| Data handling             | pandas                    | pip install pandas                              | Filtering and CSV/JSON export for the dashboard's historical log view.                                                                                                    |
| Backend / API             | FastAPI + uvicorn         | pip install fastapi<br>uvicorn python-multipart | Async-native server exposing JSON endpoints; reuses the same pydantic schemas for request/response validation; free interactive docs at <code>/docs</code> .              |
| Dashboard frontend        | HTML / CSS / vanilla JS   | —                                               | Static pages using <code>fetch()</code> and <code>setInterval()</code> to poll the FastAPI endpoints and render the three required views.                                 |
| Config                    | python-dotenv             | pip install python-dotenv                       | Keep API keys out of source code.                                                                                                                                         |

## 2.2 Why This Stack

Two of the four behavior classes are state-based conditions (an open panel persists across many frames with no action required) while two are action-based events (a person stepping off the walkway, a forklift carrying too much). Pretrained detectors and the Gemini API's multimodal reasoning cover both cases without any model training, which matters most given a multi-day, not multi-week, deadline. SQLite keeps storage file-based with zero setup. For the dashboard, a small FastAPI backend exposes plain JSON endpoints — and reuses the same pydantic models you already wrote for the rule and report schemas, so request/response validation comes for free. The frontend is

static HTML/CSS/JavaScript that polls those endpoints with `fetch()`, which keeps the server/UI split thin without pulling in a separate frontend framework or build step.

## 3. Environment Setup

### 3.1 Folder Structure

```
factory-compliance-system/
  README.md                                # setup, architecture, parsing & severity
  rationale, limitations
  compliance_policy.pdf                    # copy of the provided policy document
  requirements.txt                         # pinned dependencies
  .env                                    # GEMINI_API_KEY (never committed)
  .gitignore                              # venv/, .env, __pycache__/, outputs/*.db
  data/
    clip_001.mp4                          # provided raw clips
    clip_002.mp4
    ...
  src/
    __init__.py
    config.py                             # loads .env; shared constants & thresholds
    policy_parsing/
      __init__.py
      pdf_text_extractor.py               # extract_text_from_pdf()
      rule_extractor.py                   # parse_rules_with_gemini()
      schema.py                           # PolicyRule, SeveritySignal models
      verify_ground_truth.py              # diffs parser output vs ground truth
    detection/
      __init__.py
      video_utils.py                     # frame sampling / clip loading (OpenCV)
      walkway_detector.py                 # Class 0 - Safe Walkway Violation
      vest_detector.py                    # Class 1 - Unauthorized Intervention
      panel_detector.py                   # Class 2 - Opened Panel Cover (Gemini)
      forklift_detector.py                # Class 3 - Carrying Overload
      run_detection.py                    # orchestrates all 4 detectors per clip
    severity/
      __init__.py
      classify_severity.py                 # behavior_class -> LOW/MED/HIGH/CRIT
    escalation/
      __init__.py
      alert_queue.py                      # in-process queue for HIGH/CRITICAL
      route_event.py                     # LOW/MED -> DB; HIGH/CRIT -> DB + alert
    reports/
      __init__.py
      schema.py                           # ComplianceReport model
      db.py                               # sqlite connection + insert/query helpers
      export.py                           # CSV/JSON export for the dashboard
    dashboard/
      __init__.py
      main.py                             # FastAPI app + all /api routes
      static/
        index.html                       # the 3 required dashboard views
        style.css
        script.js                         # fetch() + setInterval() polling logic
    run_pipeline.py                       # detect -> severity -> escalate -> report
```

```

outputs/
  policy_rules.json      # generated rules (Module 1 input)
  reports.db             # sqlite log (or reports.json / reports.csv)
tests/                  # optional but recommended
  test_policy_parsing.py
  test_severity.py

```

## 3.2 Create the Environment

```

python -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate
pip install -r requirements.txt

```

## 3.3 requirements.txt

```

opencv-python
ultralytics
google-genai
pdfplumber
pydantic
pandas
fastapi
uvicorn
python-multipart
python-dotenv

```

## 3.4 API Keys

Create a .env file in the repo root with your Gemini API key, and load it with python-dotenv at the top of any script that calls the Gemini API. Never commit this file — add it to .gitignore.

```
GEMINI_API_KEY=your_key_here
```

# 4. Step-by-Step Build Walkthrough

Build a thin path from end to end before making any single module fancy. A rough pipeline that runs beginning-to-end scores far better than one polished module sitting next to four empty folders, since the assignment is graded as one integrated system.

## 4.1 Day 1 — Build the Thin End-to-End Skeleton

Goal for today: run one command and see one real detection appear on a dashboard page, however ugly.

1. Set up the folder structure, virtual environment, and dependencies (Section 3).
2. Use pdfplumber to dump the policy PDF's text to a plain string.
3. Write one prompt to the Gemini API (google-genai SDK) that extracts a single rule — the walkway rule — into the JSON schema in Appendix A. Read the output back and sanity-check it by eye.

4. Pick one short clip. Use OpenCV to grab a frame, run YOLOv8 to detect a person, manually define the green walkway boundary as a polygon of (x, y) points for that camera angle, and check whether the detected person's feet fall inside or outside it.
5. Write that one result as a single row into a new SQLite table, using the report fields from Section 5.5.
6. Write one FastAPI route that returns that row as JSON — test it directly via the auto-generated /docs page — then a single static index.html page with a short <script> that fetches it and prints the row on screen.

## 4.2 Day 2 — Widen to the Full Pipeline

Goal for today: every module handles all four behavior classes, across every provided clip.

7. Extend the extraction prompt to pull all four rules from the PDF in one pass; save the result as policy\_rules.json.
8. Compare that output, field by field, against the hand-extracted ground-truth file in Appendix B, and note any mismatches in your README's verification section.
9. Implement the walkway detector for every clip using the polygon-containment approach from Day 1.
10. Implement the vest detector: crop the bounding box around a person's torso when they're near equipment, and classify the dominant color as green vs. red-black.
11. Implement the panel-cover detector: crop the panel region and ask the Gemini API whether the cover is open or closed.
12. Implement the forklift block-count detector: count blocks on the forks (YOLOv8 object counting, or a Gemini API prompt), and attach a confidence score to the count.
13. Implement the severity classifier as a plain Python function mapping behavior\_class to a tier, using the rationale in Section 5.3.
14. Implement the escalation pipeline: always write to SQLite; additionally push a message onto an in-process queue when the tier is HIGH or CRITICAL.
15. Implement the full report generator so every field in Section 5.5 is populated for every detected event.
16. Build out the FastAPI endpoints and the three HTML/CSS/JS dashboard views described in Section 5.6, with the frontend polling the API on an interval.

## 4.3 Day 3 — Handle Edge Cases, Document, and Submit

Goal for today: a defensible, well-documented submission, not a perfect one.

17. Add a confidence threshold to the forklift counter — when it's below your threshold, flag the event for human review instead of asserting a severity tier.

18. Confirm that a clip with two different violations produces two independent events rather than one merged event.
19. Run the full pipeline across every provided clip and skim the outputs for obviously wrong detections.
20. Fill in the README using the skeleton: setup instructions, policy-parsing approach and verification, severity rationale tied to specific PDF language, and an honest limitations section.
21. Record a short demo walkthrough video if time allows — it's marked optional, which is exactly why doing it stands out.
22. Push a final commit, confirm the repository is accessible to reviewers, and submit the link through the required form before the deadline.

## 5. Module Deep-Dive: Technical Reference

### 5.1 Policy Parsing Module

Feed the full extracted PDF text to the Gemini API (google-genai SDK) in one prompt. Gemini supports structured output directly — passing `response_mime_type="application/json"` together with a `response_schema` enforces the schema in Appendix A at the API level, which is more reliable than asking for JSON in plain prompt text. Keep temperature low to reduce drift, and require a `policy_section_ref` and a short `source_excerpt` on every rule, so each extracted rule is traceable back to the document.

**Verification:** Diff the parser's output against the hand-built ground-truth file (Appendix B) field by field. Where they disagree, that's either a bug in your prompt or a place where the policy language was genuinely ambiguous — document either outcome in the README rather than silently picking one.

#### 5.1.1 Route Design: Logic First, Then Routes

Write the extraction logic as plain functions first, and test them on their own before wrapping anything in a FastAPI route. A function you can call directly and print the output of is far easier to debug than an endpoint, since a broken endpoint could mean the PDF parsing, the Gemini call, or FastAPI itself — untangling which is harder once they're all tied together.

- `extract_text_from_pdf(file_bytes) -> str` — pdfplumber, tested standalone against the real policy PDF.
- `parse_rules_with_gemini(text) -> list[PolicyRule]` — tested standalone, output compared to Appendix B.
- Only once both work on their own: wrap them in a route.

There are two reasonable ways to expose this. Option A treats the policy PDF as a fixed file shipped with the repo, parsed once when the server starts — simplest, and matches how the assignment frames the policy document as a given input:

```

from fastapi import FastAPI

app = FastAPI()
POLICY_RULES = []

@app.on_event("startup")
def load_policy():
    global POLICY_RULES
    text = extract_text_from_pdf(open("compliance_policy.pdf", "rb").read())
    POLICY_RULES = parse_rules_with_gemini(text)

@app.get("/api/policy/rules")
def get_rules():
    return POLICY_RULES

```

Option B adds a real upload endpoint instead, which makes a nicer live demo since a reviewer can watch the system ingest the PDF rather than trusting it happened offline:

```

from fastapi import FastAPI, UploadFile, File

app = FastAPI()

@app.post("/api/policy/parse")
async def parse_policy(file: UploadFile = File(...)):
    contents = await file.read()
    text = extract_text_from_pdf(contents)
    rules = parse_rules_with_gemini(text)
    return {"rules": rules}

```

*Start with Option A to get the full pipeline working end-to-end fast, and only add Option B near the end if time allows — it's the same underlying functions behind a different entry point, so it's a small addition once the logic already works.*

## 5.2 Detection Engine — Per-Class Technique

| Behavior Class                  | Type         | Technique                                            | Tooling                         |
|---------------------------------|--------------|------------------------------------------------------|---------------------------------|
| Safe Walkway Violation          | Action-based | Polygon-containment check on person bounding box     | OpenCV + YOLOv8                 |
| Unauthorized Intervention       | Action-based | Crop person bounding box, classify vest color        | OpenCV color mask or Gemini API |
| Opened Panel Cover              | State-based  | Per-frame open/closed classification of panel region | Gemini API (multimodal prompt)  |
| Carrying Overload with Forklift | Action-based | Count blocks on forks, attach confidence score       | YOLOv8 counting or Gemini API   |

## 5.3 Severity Mapping Rationale

One defensible mapping, built from the policy's own WARNING vs. CRITICAL SAFETY NOTICE callouts and whether each behavior is state-based or action-based, is shown below. Treat this as a starting point, not a fixed answer — your README should state your own reasoning in your own words.

| Behavior                        | Section / Callout              | Suggested Tier | Rationale                                                                                                       |
|---------------------------------|--------------------------------|----------------|-----------------------------------------------------------------------------------------------------------------|
| Safe Walkway Violation          | 3.3.2 — WARNING                | MEDIUM         | Action-based with personnel present, but flagged only as high-frequency, not as the highest-consequence hazard. |
| Unauthorized Intervention       | 4.3.2 — CRITICAL SAFETY NOTICE | HIGH           | Direct, immediate injury risk to the person performing the intervention.                                        |
| Opened Panel Cover              | 5.2.2 — WARNING                | LOW            | Explicitly state-based; the policy flags it as a condition, with no concurrent personnel exposure required.     |
| Carrying Overload with Forklift | 6.3.2 — CRITICAL SAFETY NOTICE | CRITICAL       | Vehicle instability can endanger bystanders beyond the operator; policy language demands an immediate alert.    |

## 5.4 Escalation Pipeline

Routing is binary and should stay simple:

- LOW / MEDIUM: write a structured row to SQLite. No alert.
- HIGH / CRITICAL: write the same row to SQLite, and also push a message onto an in-process queue (a plain Python list or queue.Queue is enough) that the dashboard polls to trigger a visible banner or flash.
- Multiple simultaneous violations in one clip: log each as its own independent event rather than merging them into one record.

## 5.5 Report Generator — Required Fields

| Field     | Description                                 | Format                    |
|-----------|---------------------------------------------|---------------------------|
| event_id  | Unique identifier for this violation event. | UUID or comparable string |
| timestamp | Wall-clock time the violation was detected. | ISO 8601                  |

|                   |                                       |                                |
|-------------------|---------------------------------------|--------------------------------|
| clip_id           | Identifier of the source video clip.  | Filename or reference          |
| zone              | Location/zone within the facility.    | String label                   |
| behavior_class    | Which of the four unsafe behaviors.   | One of four classes            |
| policy_rule_ref   | Policy section that defines the rule. | e.g. "Section 4.3.2"           |
| event_description | What was observed, in plain text.     | Free text, 1+ sentence         |
| severity          | Tier assigned by Module 2.            | LOW / MEDIUM / HIGH / CRITICAL |
| escalation_action | Routing action taken by Module 3.     | Free text                      |

## 5.6 Dashboard Views

The dashboard is a small FastAPI app exposing JSON endpoints (e.g. `/api/live`, `/api/events`, `/api/log`), paired with static HTML/CSS pages and vanilla JavaScript. FastAPI's auto-generated docs at `/docs` make each endpoint testable on its own before any frontend code exists. The frontend polls those endpoints with `fetch()` on a `setInterval()` timer to stay near-real-time; Server-Sent Events are a reasonable upgrade later if there's time, but polling is enough to satisfy the brief.

- Live Feed Monitor — shows the processed clip with a status indicator (no violation / violation + severity color / alert active), and renders the real-time alert for HIGH or CRITICAL events.
- Alert Timeline Stream — a running chronological list of events as they're detected.
- Historical Log & Export — the full log, filterable by date range, severity, and behavior class, with a working export button (JSON or CSV).

## 6. Appendix

### 6.1 Ground-Truth Extracted Rules (Reference)

Hand-extracted from the policy manual, for verifying your own parser's output against — see Section 5.1.

| Class | Domain                | Unsafe Behavior           | Observable Indicator                                            |
|-------|-----------------------|---------------------------|-----------------------------------------------------------------|
| 0     | Pedestrian Movement   | Safe Walkway Violation    | Person's position relative to the green floor-marking boundary. |
| 1     | Equipment Interaction | Unauthorized Intervention | Color of the safety vest worn while touching equipment.         |
| 2     | Electrical Safety     | Opened Panel Cover        | Open/closed visual state of the panel cover.                    |

|   |               |                                 |                                            |
|---|---------------|---------------------------------|--------------------------------------------|
| 3 | Forklift Load | Carrying Overload with Forklift | Count of standardized blocks on the forks. |
|---|---------------|---------------------------------|--------------------------------------------|

## 6.2 Submission Checklist

| Item                                                                              | Done? |
|-----------------------------------------------------------------------------------|-------|
| All five modules wired together end-to-end                                        |       |
| Pipeline runs without crashing across every provided clip                         |       |
| A compliance report is generated for every detected violation                     |       |
| Dashboard shows all three required views                                          |       |
| README covers setup, policy-parsing approach, severity rationale, and limitations |       |
| Acknowledgement email sent within 24 hours of receiving the assessment            |       |
| Demo video recorded and linked (optional)                                         |       |
| GitHub link and details submitted via the required form before the deadline       |       |